

LAPORAN
MATA KULIAH PENGENALAN POLA



Disusun Oleh:

Sindu Aditya Janadi 24/554314/NPA/19976

M. Alauddin Nuurul Hudaa 24/554313/NPA/19975

Jovan Santosa 24/554311/NPA/19973

DEPARTEMEN ELEKTRONIKA DAN INSTRUMENTASI
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA

YOGYAKARTA
2025

Pendahuluan

Perkembangan teknologi informasi yang pesat telah mendorong pertumbuhan volume teks digital dalam berbagai bentuk, termasuk artikel ilmiah, ulasan pengguna, berita, dan media sosial. Dalam konteks ini, analisis teks menjadi metode penting untuk menggali informasi tersembunyi yang dapat mendukung pengambilan keputusan, penelitian, maupun pengembangan sistem berbasis kecerdasan buatan. Salah satu aspek penting dari analisis teks adalah pengukuran kemiripan antar dokumen untuk mengetahui seberapa relevan atau serupa dua atau lebih dokumen satu sama lain.

Pengukuran kemiripan dokumen secara otomatis dapat dilakukan dengan pendekatan berbasis representasi vektor. Salah satu metode populer dalam representasi ini adalah Term Frequency-Inverse Document Frequency (TF-IDF), yang mengukur kepentingan suatu kata dalam sebuah dokumen relatif terhadap kumpulan dokumen lainnya. Setelah teks direpresentasikan sebagai vektor TF-IDF, kemiripan antar dokumen dapat dihitung menggunakan cosine similarity, yaitu ukuran kesamaan sudut antara dua vektor dalam ruang multidimensi.

Dalam tugas ini, dilakukan analisis kemiripan terhadap tiga abstrak jurnal ilmiah yang diambil dari situs Indonesian Journal of Computing and Cybernetics Systems (IJCCS) milik Universitas Gadjah Mada. Ketiga abstrak tersebut memiliki topik yang berbeda, namun masih berada dalam ruang lingkup yang serupa, yaitu implementasi teknologi kecerdasan buatan dan analisis data. Analisis ini bertujuan untuk menghitung tingkat kemiripan antar ketiga abstrak tersebut berdasarkan bobot TF-IDF dan mengukur jarak kemiripannya menggunakan metode cosine similarity. Hasil dari tugas ini diharapkan dapat memberikan gambaran tentang bagaimana dokumen yang berbeda dapat diukur kesamaannya secara kuantitatif menggunakan metode tekstual.

Deskripsi Data

No	Judul Jurnal	Link Jurnal
1	Prediction Sentiment Analysis Grab Reviews Using SVM Linear Based Streamlit	https://jurnal.ugm.ac.id/ijccs/article/view/104924
2	Multi-Domain Sentiment Analysis on Ibu Kota Nusantara (IKN) Tweets Using CNN-LSTM	https://jurnal.ugm.ac.id/ijccs/article/view/104925
3	SMOTE-SVM for Handling Imbalanced Data in Obesity Classification	https://jurnal.ugm.ac.id/ijccs/article/view/103994

Prediction Sentiment Analysis Grab Reviews Using SVM Linear Based Streamlit		
No	Kata	Jumlah Frekuensi

1	Sentimen	6 Kali
2	Prediction	4 Kali
3	Streamlit	4 Kali
4	Model	4 Kali
5	Grab	3 Kali
6	Reviews	3 Kali
7	Svm	2 Kali
8	Linear	2 Kali
9	Feature	2 Kali
10	Visualization	2 Kali
11	User	2 Kali
12	Evaluation	2 Kali
13	Accuracy	2 Kali

Multi-Domain Sentiment Analysis on Ibu Kota Nusantara (IKN) Tweets Using CNN-LSTM

No	Kata	Jumlah Frekuensi
1	Classification	3 Kali
2	Data	3 Kali
3	Balancing	3 Kali
4	Accuracy	2 Kali
5	Techniques	2 Kali
6	Performance	2 Kali
7	Model	2 Kali
8	Smote	2 Kali
9	Obesity	2 Kali
10	Health	2 Kali

11	Linear	2 Kali
12	Minority	2 Kali
13	Study	2 Kali

SMOTE-SVM for Handling Imbalanced Data in Obesity Classification		
No	Kata	Jumlah Frekuensi
1	Model	6 Kali
2	Sentiment	4 Kali
3	Analysis	4 Kali
4	Multi-Domain	4 Kali
5	Performance	3 Kali
6	Cnn-Lstm	2 Kali
7	Lstm	2 Kali
8	Cnn	2 Kali
9	Domain	2 Kali
10	Ikn	2 Kali
11	Project	2 Kali
12	Public	2 Kali
13	Opinions	2 Kali
14	Characteristics	2 Kali

Perhitungan TF-IDF

1. Daftar Term (Kata) yang Digunakan

sentiment, prediction, streamlit, model, grab, reviews, svm, linear, feature, visualization, user, evaluation, accuracy, classification, data, balancing, techniques, performance, SMOTE, obesity, health, minority, study, analysis, multi-domain, CNN-LSTM, LSTM, CNN, domain, IKN, project, public, opinions, characteristics

Total term unik: **35**

2. Menghitung Frekuensi kata / term

Frekuensi menunjukkan berapa kali kata muncul di masing-masing dokumen.

Contoh:

- Kata **sentiment** muncul 6 kali di Doc1, 0 kali di Doc2, 4 kali di Doc3.
- Kata **Prediction** muncul 4 kali pada Doc1, 0 kali di Doc 2, 0 kali di Doc3

3. Perhitungan TF (Term Frequency)

Rumus:

$$TF(t, d) = \frac{\text{Jumlah kemunculan term } t \text{ di dokumen } d}{\text{Total semua term dalam dokumen } d}$$

Contoh untuk Doc1:

- Total kemunculan semua term: $6 + 4 + 4 + 4 + 3 + 3 + 2 \times 7 = 38$
- $TF(\text{sentiment}) = 6 / 38 \approx 0.1579$
- $TF(\text{prediction}) = 4 / 38 \approx 0.1053$
- $TF(\text{grab}) = 3 / 38 \approx 0.0789$

Lakukan hal yang sama untuk Doc2 dan Doc3.

4. Perhitungan DF (Document Frequency)

Term	DF	Keterangan
sentiment	2	Muncul di Doc1 dan Doc3
prediction	1	Muncul hanya di Doc1
streamlit	1	Muncul hanya di Doc1
model	3	Muncul di ketiga dokumen (Doc1, Doc2, Doc3)
grab	1	Muncul hanya di Doc1
reviews	1	Muncul hanya di Doc1
svm	1	Muncul hanya di Doc1
linear	1	Muncul hanya di Doc1
feature	1	Muncul hanya di Doc1
visualization	1	Muncul hanya di Doc1
user	1	Muncul hanya di Doc1
evaluation	1	Muncul hanya di Doc1
accuracy	2	Muncul di Doc1 dan Doc2
classification	1	Muncul hanya di Doc2
data	1	Muncul hanya di Doc2
balancing	1	Muncul hanya di Doc2
techniques	1	Muncul hanya di Doc2
performance	2	Muncul di Doc2 dan Doc3
SMOTE	1	Muncul hanya di Doc2
obesity	1	Muncul hanya di Doc2

health	1	Muncul hanya di Doc2
minority	1	Muncul hanya di Doc2
Term	DF	Keterangan
study	1	Muncul hanya di Doc2
analysis	1	Muncul hanya di Doc3
multi-domain	1	Muncul hanya di Doc3
CNN-LSTM	1	Muncul hanya di Doc3
LSTM	1	Muncul hanya di Doc3
CNN	1	Muncul hanya di Doc3
domain	1	Muncul hanya di Doc3
IKN	1	Muncul hanya di Doc3
project	1	Muncul hanya di Doc3
public	1	Muncul hanya di Doc3
opinions	1	Muncul hanya di Doc3
characteristics	1	Muncul hanya di Doc3

5. Perhitungan IDF (Inverse Document Frequency)

Rumus :

$$IDF(t) = \log \left(\frac{N + 1}{DF(t) + 1} \right) + 1$$

Dengan:

- N = jumlah dokumen (3)
- +1 digunakan untuk smoothing agar tidak terjadi pembagian dengan nol

Contoh:

- $IDF(\text{sentiment}) = \log((3 + 1)/(2 + 1)) + 1 = \log(4/3) + 1 \approx \mathbf{1.1249}$
- $IDF(\text{prediction}) = \log(4/1 + 1) = \log(2) + 1 \approx \mathbf{1.6931}$
- $IDF(\text{model}) = \log(4/4) + 1 = \log(1) + 1 = \mathbf{1.0}$

6. Perhitungan TF-IDF

Rumus :

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

Contoh untuk Doc1:

- $TF(\text{sentiment}) \approx 0.1579, IDF \approx 1.1249 \Rightarrow TF\text{-}IDF \approx 0.1777$
- $TF(\text{prediction}) \approx 0.1053, IDF \approx 1.6931 \Rightarrow TF\text{-}IDF \approx 0.1782$
- $TF(\text{model}) \approx 0.1053, IDF = 1.0 \Rightarrow TF\text{-}IDF = 0.1053$

Cosine Similarity

Cosine Similarity adalah ukuran kesamaan antara dua vektor non-nol dari ruang hasil kali dalam yang mengukur kosinus sudut di antara keduanya. Semakin dekat nilai kosinus ke 1, semakin mirip kedua vektor tersebut. Dalam konteks pemrosesan bahasa alami, ini digunakan untuk mengukur kesamaan antara dua dokumen dengan merepresentasikan dokumen sebagai vektor (misalnya, vektor TF-IDF). Formula Cosine Similarity antara dua vektor v_1 dan v_2 adalah :

$$\text{similarity} = \cos(\theta) = \frac{v_1 \cdot v_2}{||v_1|| \cdot ||v_2||}$$

Di mana:

- $v_1 \cdot v_2$ adalah hasil perkalian dot (dot product) dari kedua vektor.
- dan $||v_2||$ adalah norma (magnitude) Euclidean dari masing-masing vektor.

Fungsi *cosine_similarity(v1, v2)* dalam kode menghitung nilai ini.

Implementasi Program

Implementasi Program pada proyek ini berfokus pada proses perhitungan *cosine similarity* antar dokumen menggunakan metode *Term Frequency-Inverse Document Frequency* (TF-IDF). Tahap ini bertujuan untuk merealisasikan logika perhitungan kemiripan teks ke dalam bentuk kode yang dapat dijalankan, sehingga dapat digunakan untuk mengukur sejauh mana dua atau lebih dokumen memiliki kesamaan isi.

Link Google Colab : [TugasPP14Mei.ipynb](https://colab.research.google.com/drive/1TugasPP14Mei.ipynb)

1. Import Library

```
import math
```

Pada proyek ini, peneliti tidak menggunakan banyak library, peneliti berfokus pada pengimplementasian algoritma untuk menghitung hasil TF-IDF dan Cosine Similarity secara eksplisit.

2. Mendefinisikan Terms

```
terms = [  
    "sentiment", "prediction", "streamlit", "model", "grab", "reviews",  
    "svm", "linear", "feature", "visualization", "user", "evaluation", "accuracy",  
    "classification", "data", "balancing", "techniques", "performance",  
    "SMOTE", "obesity", "health", "minority", "study",  
    "analysis", "multi-domain", "CNN-LSTM", "LSTM", "CNN", "domain",  
    "IKN", "project", "public", "opinions", "characteristics"  
]
```

Pada tahap ini didefinisikan “terms” sebagai list berisi kata - kata atau istilah penting yang memiliki frekuensi kemunculan lebih dari sekali pada ketiga abstrak.

3. Mengisi Nilai Frekuensi

```
doc1 = [6, 4, 4, 4, 3, 3, 2, 2, 2, 2, 2, 2, 2,  
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0]  
  
doc2 = [0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 2,  
        3, 3, 3, 2, 2, 2, 2, 2, 2, 2,  
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0]  
  
doc3 = [4, 0, 0, 6, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
        0, 0, 0, 0, 3, 0, 0, 0, 0, 0,  
        4, 4, 2, 2, 2, 2, 2, 2, 2, 2]  
  
documents = [doc1, doc2, doc3]  
N = len(documents)
```

Nilai frekuensi dari setiap term dalam masing - masing dokumen diisi dalam bentuk list. Kumpulan nilai - nilai tersebut ditulis secara berurutan sesuai dengan urutan kata atau istilah penting yang telah didefinisikan sebelumnya.

4. Perhitungan Term Frequency

```
def compute_tf(doc):
    total_terms = sum(doc)
    return [count / total_terms if total_terms != 0 else 0 for count in doc]

tf_docs = [compute_tf(doc) for doc in documents]
```

Pada tahap ini, dibuat sebuah fungsi bernama `compute_tf` yang bertugas untuk menghitung frekuensi relatif dari setiap kata atau istilah penting dalam dokumen. Fungsi ini membagi jumlah kemunculan kata tertentu dengan total jumlah kata dalam dokumen sehingga menghasilkan nilai TF dalam bentuk proporsi. Perhitungan dilakukan untuk setiap dokumen dengan menggunakan list comprehension sehingga menghasilkan daftar vektor TF yang mewakili setiap dokumen.

5. Perhitungan Document Frequency

```
df = []
for i in range(len(terms)):
    count = sum(1 for doc in documents if doc[i] > 0)
    df.append(count)
```

Pada tahap ini dilakukan perhitungan nilai Document Frequency dengan cara melakukan iterasi terhadap setiap indeks term, lalu menghitung berapa banyak dokumen yang memiliki nilai frekuensi lebih dari nol untuk term tersebut, kemudian nilai - nilai yang dihasilkan disimpan ke dalam list `df`.

6. Perhitungan Inverse Document Frequency

```
idf = [math.log((N + 1) / (df_i + 1)) + 1 for df_i in df] # +1 smoothing
```

Perhitungan ini menggunakan rumus $idf = \log((N + 1) / (df + 1)) + 1$, di mana N adalah jumlah total dokumen dan df adalah jumlah dokumen yang mengandung term tertentu. Penambahan nilai 1 pada pembilang bertujuan untuk menghindari pembagian dengan nol serta memperhalus hasil perhitungan.

7. Perhitungan Term Frequency-Inverse Document Frequency

```

tfidf_docs = []
for tf in tf_docs:
    tfidf = [tf[i] * idf[i] for i in range(len(terms))]
    tfidf_docs.append(tfidf)

```

Perhitungan dilakukan dengan mengalikan nilai TF dari setiap term dalam dokumen dengan nilai IDF term tersebut. Hal ini bertujuan untuk memberi bobot yang lebih tinggi pada term yang sering muncul di suatu dokumen namun jarang muncul di dokumen lain, sehingga lebih representatif dalam membedakan isi antar dokumen. Hasil perhitungan TF-IDF dari setiap dokumen disimpan dalam list `tfidf_docs` sebagai representasi vektor berbobot untuk keperluan analisis lebih lanjut yaitu cosine similarity.

8. Perhitungan Cosine Similarity

```

def cosine_similarity(v1, v2):
    dot = sum(a * b for a, b in zip(v1, v2))
    norm1 = math.sqrt(sum(a * a for a in v1))
    norm2 = math.sqrt(sum(b * b for b in v2))
    return dot / (norm1 * norm2) if norm1 != 0 and norm2 != 0 else 0

```

Pembuatan fungsi mengimplementasikan rumus cosine similarity, yaitu membagi hasil dot product (perkalian dan penjumlahan elemen yang bersesuaian) dari dua vektor dengan hasil perkalian norma (panjang) masing-masing vektor. Fungsi ini juga dilengkapi dengan penanganan kasus khusus, yaitu jika salah satu norma bernilai nol (yang berarti vektor kosong), maka hasilnya otomatis diberikan nilai nol untuk menghindari pembagian dengan nol. Hasil dari fungsi ini menunjukkan seberapa mirip dua dokumen, dengan nilai berkisar antara 0 (tidak mirip) hingga 1 (sangat mirip).

```

sim_1_2 = cosine_similarity(tfidf_docs[0], tfidf_docs[1])
sim_1_3 = cosine_similarity(tfidf_docs[0], tfidf_docs[2])
sim_2_3 = cosine_similarity(tfidf_docs[1], tfidf_docs[2])

```

program menghitung nilai cosine similarity antar pasangan dokumen berdasarkan vektor TF-IDF yang telah dibentuk sebelumnya. Nilai kemiripan dihitung antara Dokumen 1 dengan Dokumen 2 (sim_1_2), Dokumen 1 dengan Dokumen 3 (sim_1_3), serta Dokumen 2 dengan Dokumen 3 (sim_2_3) dengan menggunakan fungsi cosine_similarity.

9. Mencetak Hasil Perhitungan Cosine Similarity berdasarkan TF-IDF

```
print("Cosine Similarity:")
print(f"Doc 1 vs Doc 2: {sim_1_2:.4f}")
print(f"Doc 1 vs Doc 3: {sim_1_3:.4f}")
print(f"Doc 2 vs Doc 3: {sim_2_3:.4f}")
```

Program mencetak nilai kemiripan antara setiap pasangan dokumen dengan format desimal empat angka di belakang koma. Output ini memberikan gambaran langsung mengenai seberapa mirip masing-masing dokumen satu sama lain berdasarkan bobot TF-IDF yang telah dihitung sebelumnya. Dengan demikian, pengguna dapat dengan mudah menginterpretasikan hubungan antar dokumen berdasarkan skor kemiripannya.

```
matrix = [
    [1.00, sim_1_2, sim_1_3],
    [sim_1_2, 1.00, sim_2_3],
    [sim_1_3, sim_2_3, 1.00]
]

# Header dan index
headers = ["Doc 1", "Doc 2", "Doc 3"]
print("Cosine Similarity Matrix:")
print("    " + " ".join(f"{h:>7}" for h in headers))
for i, row in enumerate(matrix):
    row_str = " ".join(f"{val:7.4f}" for val in row)
    print(f"{headers[i]:<7} {row_str}")
```

Untuk mempermudah pembacaan hasil, output dirancang agar menampilkan hasil dalam bentuk matriks. Matriks ini disusun dengan diagonal utama bernilai 1, karena setiap dokumen pasti memiliki kemiripan maksimal dengan dirinya sendiri. Nilai-nilai di luar diagonal menunjukkan tingkat kemiripan antara pasangan dokumen yang berbeda. Matriks ini kemudian dicetak dengan menyesuaikan lebar kolom agar setiap nilai kemiripan ditampilkan dengan presisi empat angka di belakang koma. Dengan cara ini, hasil perhitungan cosine similarity antara dokumen-dokumen dapat dengan jelas terlihat dan dianalisis dalam bentuk tabel.

Hasil dan Analisis

```
PS D:\Kuliah\Semester 4 (UGM)\Pengenalan Pola\Similarity> python .\model.py
Cosine Similarity:
Doc 1 vs Doc 2: 0.0667
Doc 1 vs Doc 3: 0.2253
Doc 2 vs Doc 3: 0.1040
PS D:\Kuliah\Semester 4 (UGM)\Pengenalan Pola\Similarity> []
```

- **Doc 1 vs Doc 2 (0.0667)** : Nilai similarity yang sangat rendah ini menunjukkan bahwa Dokumen 1 dan Dokumen 2 memiliki konten yang sangat berbeda. Hal ini konsisten dengan observasi sebelumnya mengenai perbedaan distribusi term di antara kedua dokumen tersebut.
- **Doc 1 vs Doc 3 (0.2253)** : Nilai similarity ini lebih tinggi dibandingkan perbandingan Doc 1 vs Doc 2 dan Doc 2 vs Doc 3, menunjukkan adanya tingkat kesamaan konten antara Dokumen 1 dan Dokumen 3. Meskipun nilainya tidak tinggi, ini mengindikasikan bahwa ada beberapa term atau topik yang tumpang tindih di antara kedua dokumen ini.
- **Doc 2 vs Doc 3 (0.1040)** : Nilai similarity yang rendah ini menunjukkan bahwa Dokumen 2 dan Dokumen 3 memiliki konten yang paling tidak mirip di antara ketiga pasangan. Ini juga konsisten dengan observasi sebelumnya mengenai perbedaan fokus topik antara kedua dokumen tersebut.

Secara keseluruhan, hasil Cosine Similarity yang diperbarui ini masih menunjukkan pola yang sama: Dokumen 1 dan Dokumen 3 memiliki kesamaan konten yang paling tinggi di antara ketiga pasangan, meskipun nilai similarity-nya moderat. Dokumen 1 dan Dokumen 2, serta Dokumen 2 dan Dokumen 3, memiliki kesamaan konten yang sangat rendah.

Kesimpulan

Berdasarkan analisis kemiripan antar tiga abstrak jurnal ilmiah menggunakan metode TF-IDF dan cosine similarity, diperoleh gambaran bahwa pendekatan representasi vektor mampu mengukur kesamaan dokumen secara kuantitatif. Proses ini dimulai dari ekstraksi kata-kata penting dalam setiap abstrak, perhitungan frekuensi kemunculan kata (TF), hingga pengukuran bobot tiap kata dengan mempertimbangkan frekuensi kemunculannya dalam seluruh dokumen (DF dan IDF). Hasil akhir berupa nilai cosine similarity menunjukkan tingkat kedekatan konten antar abstrak, meskipun topiknya berbeda namun berada dalam ranah yang serupa, yaitu analisis data dan kecerdasan buatan. Teknik ini terbukti efektif untuk mengenali kemiripan isi dokumen secara objektif, yang bermanfaat dalam klasifikasi dokumen, sistem rekomendasi, dan aplikasi berbasis teks lainnya.

Daftar Pustaka

- Hidayat, M. T., Arifin, M., & Muzid, S. (2025). Prediction Sentiment Analysis Grab Reviews Using SVM Linear Based Streamlit. Yogyakarta: Universitas Gadjah Mada.
- Prasastio, F. R., & Winarko, E. (2025). Multi-Domain Sentiment Analysis on Ibu Kota Nusantara (IKN) Tweets Using CNN-LSTM. Yogyakarta: Universitas Gadjah Mada.
- Biddinika, M. K., Yuliansyah, H., Soyusiawaty, D., & Razak, F. R. (2025). SMOTE-SVM for Handling Imbalanced Data in Obesity Classification. Yogyakarta: Universitas Gadjah Mada.